1 Dimensional Arrays

What is an Array?

An array is a collection of elements of the same data type stored in contiguous memory locations. In Java, arrays are objects that allow you to store multiple values of the same type under a single variable name.

Array Declaration and Initialization

Declaration Syntax

```java
dataType[] arrayName;
// or
dataType arrayName[];
```

Examples of Declaration

```java
int[] numbers;
String[] names;
char[] characters;
double[] prices;
```

Array Initialization

Method 1: Declare and Initialize Separately

```java
int[] numbers;
numbers = new int[5]; // Creates array of size 5 with default values (0)
```

Method 2: Declare and Initialize Together

```java
int[] numbers = new int[5];
```

Method 3: Initialize with Values

```java
int[] numbers = {10, 20, 30, 40, 50};
// or
int[] numbers = new int[]{10, 20, 30, 40, 50};
```

Accessing Array Elements

Array elements are accessed using index numbers starting from 0.

```java
int[] numbers = {10, 20, 30, 40, 50};

// Accessing elements
System.out.println(numbers[0]); // Output: 10
System.out.println(numbers[2]); // Output: 30

// Modifying elements
numbers[1] = 25;
System.out.println(numbers[1]); // Output: 25
```

Array Length Property

Every array in Java has a `length` property that returns the number of elements.

```java
int[] numbers = {10, 20, 30, 40, 50};
System.out.println("Array length: " + numbers.length); // Output: 5
```

Iterating Through Arrays

Using For Loop

```java
int[] numbers = {10, 20, 30, 40, 50};

for (int i = 0; i < numbers.length; i++) {
    System.out.println("Element at index " + i + ": " + numbers[i]);
}
```

Using Enhanced For Loop (For-Each)

```java
int[] numbers = {10, 20, 30, 40, 50};

for (int num : numbers) {
    System.out.println("Element: " + num);
}
```

Memory Management of Array

Memory Allocation

When you create an array in Java, memory is allocated in the heap area. The array variable itself is stored in the stack, but it contains a reference (address) to the actual array object in the heap.

```java
int[] numbers = new int[5];
```

Memory Layout:

- **Stack**: Contains the variable `numbers` which holds the reference to the array
- **Heap**: Contains the actual array object with 5 integer elements

Memory Structure

```
Stack               Heap
-------             -------
numbers -------->   [0][0][0][0][0]
(reference)         Index: 0 1 2 3 4
```

Default Values in Memory

When an array is created, elements are initialized with default values:

- `int`, `byte`, `short`, `long`: 0
- `float`, `double`: 0.0
- `boolean`: false
- `char`: '\u0000' (null character)
- Object references: null

```java
int[] numbers = new int[3];
// Memory: [0][0][0]

String[] names = new String[2];
// Memory: [null][null]
```

Memory Efficiency

Arrays store elements in contiguous memory locations, making them memory-efficient for:

- Fast access to elements using index
- Cache-friendly memory access patterns
- Minimal memory overhead per element

Array Size and Memory

The size of an array is fixed once created and cannot be changed. Memory is allocated based on:

- Data type size (int = 4 bytes, double = 8 bytes, etc.)
- Number of elements
- Additional overhead for array object metadata

```java
int[] small = new int[10];    // 40 bytes + overhead
int[] large = new int[1000];  // 4000 bytes + overhead
```

Garbage Collection

When an array is no longer referenced, it becomes eligible for garbage collection:

```java
int[] numbers = new int[100];
numbers = null; // Array becomes eligible for garbage collection
```

Common Array Operations

Finding Maximum Element

```java
int[] numbers = {45, 23, 78, 12, 67};
int max = numbers[0];

for (int i = 1; i < numbers.length; i++) {
    if (numbers[i] > max) {
        max = numbers[i];
    }
}
System.out.println("Maximum: " + max);
```

Calculating Sum

```java
int[] numbers = {10, 20, 30, 40, 50};
int sum = 0;

for (int num : numbers) {
    sum += num;
}
System.out.println("Sum: " + sum);
```

Searching for an Element

```java
int[] numbers = {10, 20, 30, 40, 50};
int target = 30;
boolean found = false;

for (int i = 0; i < numbers.length; i++) {
    if (numbers[i] == target) {
        System.out.println("Element found at index: " + i);
        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Element not found");
}
```

Quiz Questions

1. What is the correct way to declare an integer array in Java? a) int array[] = new int[5]; b) int[] array = new int[5]; c) Both a and b are correct d) int array = new int[5];

Answer: c) Both a and b are correct

2. What will be the output of the following code?

```java
int[] arr = {10, 20, 30};
System.out.println(arr.length);
```

a) 2
b) 3
c) 4
d) Compilation error

Answer: b) 3

3. Where are array objects stored in Java memory? a) Stack b) Heap c) Method area d) Program counter

Answer: b) Heap

4. What is the default value for elements in a newly created int array? a) null b) 1 c) 0 d) undefined

Answer: c) 0

5. Which statement about array indexing in Java is true? a) Array indexing starts from 1 b) Array indexing starts from 0 c) Array indexing can start from any number d) Arrays don't use indexing

Answer: b) Array indexing starts from 0

Practice Coding Questions

1. Write a Java program to create an array of 7 integers, initialize it with values of your choice, and print all elements using a for loop.

2. Create a program that finds and prints the smallest element in an integer array containing the values {45, 23, 78, 12, 67, 34}.

3. Write a Java program that creates an array of 5 strings, stores names of your friends, and prints them in reverse order.

4. Create a program that calculates and displays the average of all elements in a double array containing {12.5, 8.3, 15.7, 9.2, 11.8}.

5. Write a program that counts how many even numbers are present in an integer array {12, 7, 18, 5, 24, 13, 16, 9}.